Obserwatorium mikroklimatu

Cel i użytkownicy

Splot zbiera pomiary temperatury z wielu miejsc. Czytelnik ma publiczny dostęp do wykresu i tabeli. Administrator zarządza seriami, czujnikami i swoim hasłem. Wyniki są dopisywane wyłącznie przez API czujnika; interfejs nie oferuje ich ręcznej edycji ani usuwania.

Adresy i konto

Aplikacja i API: Nie wdrożono publicznie — do uzupełnienia
Repozytorium: Repozytorium lokalne — do uzupełnienia
Login: admin
Hasło: w lokalnym .env; wersja robocza nie zawiera sekretów

Dobór technologii

FastAPI i Pydantic zapewniają jawny kontrakt i walidację. SQLite z WAL upraszcza odtwarzanie projektu i spełnia wymaganie relacyjnej bazy; wymaga trwałego dysku jednego hosta. React jest biblioteką SPA, współdzielącą stan filtrów, wykresu i tabeli. SVG umożliwia własne znaczniki, wybór punktów oraz wydruk bez dodatkowej usługi wykresów. Generator używa standardowej biblioteki Python.

Dane i zakres

Trzy serie demonstracyjne: Szklarnia, Ogród zimowy, Taras; po 72 pomiary w stopniach Celsjusza. Generator tworzy je przez REST, a nie przez zapis do SQL. Zakresy i nazwy można zmieniać. Wyniki API są ograniczane parametrem limit; interfejs pobiera do 10 000 najnowszych w wybranym zakresie i informuje o osiągnięciu limitu.

Stan zgłoszenia

Kod, migracja, generator i lokalne testy są przygotowane. Wersja robocza wymaga uzupełnienia publicznego adresu, repozytorium, danych konta i nagrania wdrożonej aplikacji. E1: 7.11.2026; E2: 21.11.2026, 23:59. Spóźnione E1 można oddać z E2 za maksymalnie 5 pkt. Utrzymuj adres i konto do wystawienia oceny.

Uruchomienie i generator

Środowisko

Weryfikacja: Python 3.12.16, Node.js 22.23.4. Wymagane Python 3.12+, uv i Node 22.12+ z npm. Dokładne zależności: uv.lock oraz frontend/package-lock.json. Polecenia wykonuj w katalogu projekt.

Uruchomienie lokalne

```
python3 scripts/setup.py
uv sync --locked
cd frontend && npm ci && npm run build
cd ..
uv run python scripts/serve.py
```

Adres: http://127.0.0.1:8000. Drugie okno terminala:

```
uv run python scripts/seed.py
```

Setup jednorazowo generuje .env z losowym hasłem. Baza: data/splot.sqlite3. Kolejny start nie resetuje konta. Zmiana hasła w panelu unieważnia wszystkie sesje. Token UI istnieje tylko w pamięci; przeładowanie strony wymaga logowania.

Wysyłanie pomiarów

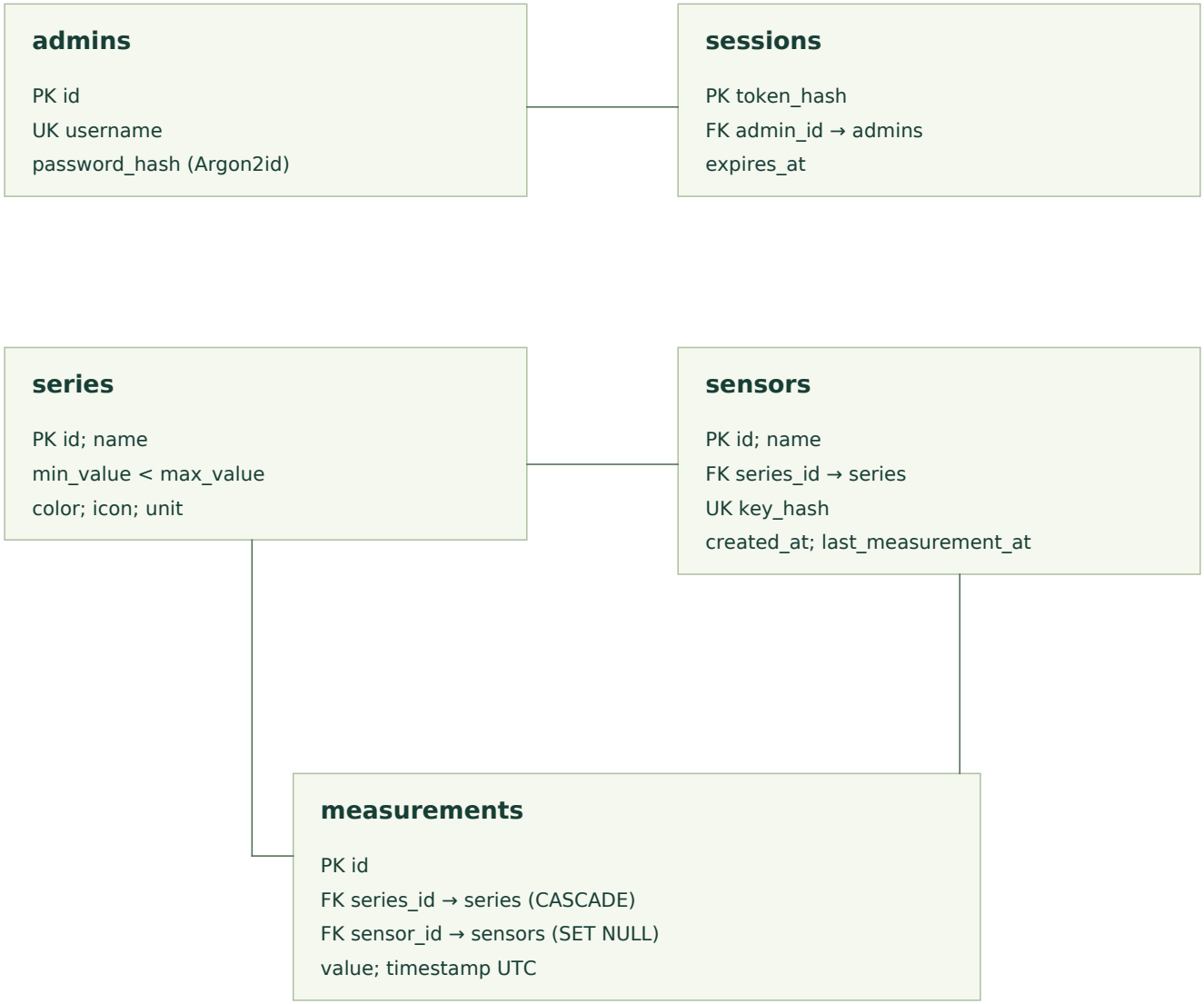
Zarejestruj czujnik w panelu. Ustaw SPLOT_API_KEY na otrzymany klucz i SPLOT_API_URL na adres bazowy, bez /api. Nie zapisuj klucza w kodzie.

```
python3 -m generator.sensor --historical --count 72 --step 1200
python3 -m generator.sensor --count 0 --interval 5 --mode random
python3 -m generator.sensor --count 1 --mode constant --value 999
```

Parametry

--api / --key lub zmienne SPLOT_API_URL / SPLOT_API_KEY; --count: liczba (0 = bez końca); --interval: odstęp wysyłania w sekundach; --mode: sinusoid, random, constant; --min / --max: zakres; --value: stała; --historical: bez czekania; --start: ISO 8601 ze strefą; --step: odstęp pomiarów historycznych; --phase / --seed: konfiguracja przebiegu. Generator wypisuje status i przyczynę odrzucenia, kończy kodem 1 przy błędach i przerywa po 401. Seed pomija istniejące serie z ≥15 wynikami, a klucze zapisuje w ignorowanym data/demo-keys.json.

Relacyjny model danych



Klucze, indeksy i spójność

Migracja: backend/migrations/001_initial.sql. Indeksy: measurements(series_id,timestamp) oraz timestamp. Czasy mają jednolity format UTC. Nie ma unikatowości czasu w serii — duplikaty czasowe zachowują własne ID. Usunięcie serii usuwa jej dane; usunięcie czujnika zeruje tylko opcjonalny FK pomiaru. BEGIN IMMEDIATE serializuje zapis i walidację. Tabele techniczne: schema_migrations, changes (rewizja SSE), login_attempts (limit prób logowania).

API i bezpieczeństwo

Wspólny kontrakt

Pełny kontrakt: vendor/zai-api-26z.yaml. /api/health; auth/login, auth/logout, auth/password; CRUD series; rejestracja, lista i wyrejestrowanie sensors; measurements: publiczny GET i POST z kluczem. Brak PUT/PATCH/DELETE pojedynczych pomiarów. Utworzenie: 201 i Location; usunięcie/logout: 204; walidacja: 400/422; brak uwierzytelnienia: 401; konflikt zakresu: 409. Błędy application/problem+json zawierają title/status/detail.

Mechanizm 1: hasła i sesje

backend/db.py używa Argon2id, funkcji przeznaczonej do haseł z losową solą. Hasło nie jest zapisywane jawnie. backend/app.py tworzy token przez secrets.token_urlsafe(32), zapisując tylko SHA-256 i termin ważności. Hasz losowego tokenu ogranicza skutki odczytu bazy. Każda operacja administracyjna sprawdza sesję. Logout usuwa token, a zmiana hasła usuwa wszystkie sesje konta. Próby logowania są limitowane. Frontend nie utrwala tokenu w localStorage.

Mechanizm 2: tożsamość czujnika

Każdy czujnik ma losowy 43-znakowy klucz. Serwer przechowuje wyłącznie jego SHA-256; jedyną odpowiedzią zawierającą jawny klucz jest rejestracja. Odczyty czujników nie ujawniają sekretów. Nagłówek X-API-Key ustala serię; seria przesłana w body nie pozwala skierować pomiaru gdzie indziej. Po usunięciu czujnika klucz natychmiast przestaje działać. Token administratora nie daje prawa zapisu pomiarów.

Pozostałe zabezpieczenia

SQL jest parametryzowany. Limit treści: 64 KB; walidacja typów, zakresu, dat i formatów. Błędy nie ujawniają stosu wywołań. CSP ogranicza źródła do tej samej domeny; X-Content-Type-Options, X-Frame-Options, Referrer-Policy i Permissions-Policy są ustawiane przez backend. HTTPS/HSTS oraz przekierowanie HTTP zapewnia docelowe wdrożenie z Caddy. Proxy headers należy akceptować wyłącznie od zaufanego reverse proxy.

Rozszerzenie SSE

GET /api/events zwraca text/event-stream. Zdarzenie update zawiera {revision: liczba}. Licznik w bazie zmienia się przy zmianach danych. Klient po zdarzeniu pobiera aktualne REST; heartbeat co 15 sekund, odczyt rewizji co 1 sekundę. EventSource wznawia połączenie. Jest to dodatkowy endpoint; wspólny kontrakt pozostaje bez zmian.

Interfejs i weryfikacja

Prezentacja

Każda seria ma osobną krzywą i kolumnę tabeli. Dane filtrowane są po seriach i domkniętym przedziale czasu. Wartość w tabeli jest przyciskiem dostępnym z klawiatury: wybór wyróżnia punkt na wykresie. Kilka wyników w jednej chwili pojawia się w tej samej komórce bez nadpisywania. Wykres ma różne kolory, znaczniki i wzory linii; opisy i tabela zapewniają alternatywę tekstową.

Responsywność, UX i druk

CSS korzysta z Flexbox, Grid i media queries. Sprawdzono widok 360 px, przewijanie samej tabeli, etykiety formularzy, fokus i Enter. Dialogi używają natywnego dialog, komunikaty aria-live/role=alert. Wykres dostosowuje viewBox do szerokości. Reguły @media print ukrywają kontrolki i drukują pełną pobraną tabelę wraz z wykresem.

Lokalne testy

Własne pytest: 23 testy, w tym zmiana hasła i sesje, trwałość po restarcie, skróty sekretów, duplikaty czasu, limit, ścisłe typy, usuwanie czujnika i równoczesna zmiana zakresu. Testy prowadzącego: E1 23/23 (10/10), E2 45/45 w grupach kontrakt/walidacja/bezpieczeństwo (13/13) na lokalnym HTTPS. Testy Playwright obejmują publiczny pulpit i cykl administracyjny, klucz, SSE, odwołanie oraz logout. Szczegóły i ograniczenia: docs/WERYFIKACJA.md.

CI i dokumentacja wyników

Workflow GitHub Actions instaluje zależności z lockfile, uruchamia pytest, build, seed, kontrakt E1 i Playwright. Kod wyjścia wrappera kontraktu zależy od wyników testów. Repozytorium nie zostało jeszcze wysłane; zielony status zdalnego CI musi być potwierdzony po push. Nie uruchomiono obrazu Docker ani testów publicznego wdrożenia. Lokalny wynik nie jest gwarancją oceny prowadzącego.

Polecenia kontrolne

uv run pytest -q
uv run python scripts/contract.py --stage E1
cd frontend && npm run test:e2e

Na publicznej domenie, z Chrome/Chromium:
uv run python scripts/contract.py --app https://DOMENA --stage E2 --lighthouse

Wdrożenie, źródła i oddanie

Publiczny serwer

Dockerfile buduje frontend, następnie uruchamia FastAPI jako użytkownik bez uprawnień root. compose.yaml i Caddyfile udostępniają porty 80/443, kierują ruch do wewnętrznego app:8000 i zachowują dane na nazwanym wolumenie. Ustaw SPLOT_DOMAIN i dane administratora w .env, wskaż DNS na serwer, uruchom docker compose up -d --build. Wprowadź dane generatorem przez publiczne API, sprawdź testy oraz trwałość po restarcie. Konfiguracji kontenerowej nie uruchomiono w środowisku implementacji.

Archiwum i demonstracja

scripts/package.py tworzy ZIP z kodem aplikacji, generatora, SQL, .env.example, testami i tym PDF. Wyklucza bazę, klucze, .env i zależności. docs/ODDANIE.md zawiera scenariusz 5–7 minut i listę kontrolną. Nagraj publiczną aplikację z paskiem adresu, pokaż F1–F13, telefon 360 px, wydruk i dwa mechanizmy bezpieczeństwa. Pełny cykl: rejestracja → klucz w generatorze → dane → 422 poza zakresem → wyrejestrowanie → 401. MP4 do 100 MB. Minuty checklisty wpisz po nagraniu.

Źródła

Wymagania i testy: materiały prowadzącego ZAI 26Z, kopie kontraktu/testów w vendor. Dokumentacja: fastapi.tiangolo.com/deployment/docker/; react.dev/reference/react/useEffect; caddyserver.com/docs/caddyfile/directives/reverse_proxy. React i Vite: MIT; ikony Lucide: ISC. Z materiałów AngularJS/GWT nie kopiowano kodu aplikacji. Pełne wersje zależności znajdują się w lockfile.

Deklaracja użycia AI

Wykorzystano asystenta OpenAI Codex do analizy wymagań, projektowania architektury, generowania i poprawiania implementacji, testów oraz dokumentacji. Zakres pomocy obejmował cały projekt. Autor zgłoszenia odpowiada za zweryfikowanie kodu, bezpieczeństwa i zgodności wymagań oraz powinien umieć wyjaśnić zastosowane rozwiązania.

Odtworzenie tego dokumentu

uv run python scripts/documentation.py

Po wdrożeniu podaj --url i --repo oraz --submission. Wersja dla prowadzącego pobiera hasło z ADMIN_PASSWORD i jest zapisywana poza Git w artifacts/dokumentacja-oddanie.pdf. Wersja robocza w docs/dokumentacja.pdf nie zawiera sekretów. Nie deklarowano nieistniejącego wdrożenia, nagrania ani zdalnego statusu CI.